INGEGNERIA DEI DATI · 2026/2027

Homework 3 — Indicizzazione e ricerca di file .txt con Lucene

Relazione sul progetto: analyzer, indicizzazione e query di test

STUDENTE
Alessio Pecilli

MATRICOLA
589232

TECNOLOGIE
Java 21 · Maven · Lucene 10.1.0

Repository GitHub: **https://github.com/Alessio-Pecilli/IDD-HW3**

1 Obiettivo e struttura del progetto

Il progetto indicizza i file `.txt` contenuti in una cartella considerando **due campi**, il **nome** del file e il **contenuto**, ciascuno con un analyzer appropriato. Un secondo programma legge le query da console, interroga l'indice e stampa i risultati.

`Indexer.java`

Apre la `Directory` (`FSDirectory`), crea un `Document` per ogni file e lo aggiunge con un `IndexWriter` , poi esegue `commit()` .

`Searcher.java`

Apre l'indice con `DirectoryReader` e `IndexSearcher` , costruisce la query con il `QueryParser` e stampa score e path dei primi 10 risultati (`TopDocs`).

`SearcherTest.java`

Test JUnit che ricreano l'indice in una cartella temporanea e verificano automaticamente i risultati delle query.

`dataset/`

12 file `.txt` in italiano su argomenti diversi (Roma, cucina, astronomia, motori di ricerca...).

Campi del documento

Campo	Tipo	Stored	Uso
`nome`	`TextField`	Sì	Nome del file senza estensione `.txt` , analizzato per cercare le singole parole
`contenuto`	`TextField`	No	Testo del file: è ricercabile anche se non viene memorizzato
`path`	`StringField`	Sì	Percorso del file, non tokenizzato; letto con `storedFields()` per stampare il risultato

Esecuzione

```
mvn compile exec:java -Dexec.mainClass=it.uniroma3.idd.Indexer   # crea l'indice in index/
mvn exec:java -Dexec.mainClass=it.uniroma3.idd.Searcher           # query da console
mvn test                                                          # test automatici
```

2 Analyzer scelti

Gli analyzer sono definiti per campo con un `PerFieldAnalyzerWrapper`, restituito da un unico metodo `Indexer.analyzer()` usato sia dall' `IndexWriterConfig` sia dal `QueryParser`: in questo modo query e documenti vengono analizzati allo stesso modo e producono termini compatibili.

Campo `nome`

CustomAnalyzer

storia_di_roma nome del file → **WhitespaceTokenizer** spezza sugli spazi → **LowerCaseFilter** tutto minuscolo → **WordDelimiterGraphFilter** spezza su `_ - .` → **token**

storia_di_roma	`storia` `di` `roma`
Ricetta Carbonara	`ricetta` `carbonara`
motori-di-ricerca	`motori` `di` `ricerca`

- **WhitespaceTokenizer**: alcuni nomi contengono spazi (`Ricetta Carbonara`).
- **LowerCaseFilter**: `Ricetta` e `ricetta` devono coincidere.
- **WordDelimiterGraphFilter**: i nomi usano `_` e `-` come separatori; il solo WhitespaceTokenizer lascerebbe `storia_di_roma` come un unico token.
- **Niente stopword né stemming**: un nome di file è breve e ogni parola conta (serve poter cercare la frase `"motori di ricerca"`); lo stemming renderebbe la ricerca sul nome troppo permissiva.
- L'estensione `.txt` viene tolta prima dell'indicizzazione, dato che è presente in tutti i file.

Campo `contenuto`

ItalianAnalyzer

impero romano e i pianeti testo del file → **ItalianAnalyzer** lowercase · stopword · stemming → **token**

impero romano e i pianeti	`imper` `roman` `pianet`

I testi sono in italiano, quindi si usa l'analyzer pre-costruito per la lingua italiana (analogo all' `EnglishAnalyzer` visto a lezione per l'inglese):

- rimuove le **stopword** italiane (`il` , `di` , `la` , `e` , ...), che non portano informazione;
- applica uno **stemming** leggero (`pianeti` , `pianeta` → `pianet`), così si trovano anche singolari/plurali e forme maschili/femminili.

3 Indicizzazione

12
file `.txt` indicizzati

~450 ms
tempo di indicizzazione (include l'avvio di Lucene)

2 + path
campi indicizzati: `nome` e `contenuto`

Il tempo è misurato in `Indexer.main` con `System.currentTimeMillis()` e comprende apertura della `Directory` , creazione dell' `IndexWriter` , lettura dei file, `addDocument` e `commit()` . Su un dataset così piccolo la maggior parte del tempo è dovuta all'avvio di Lucene e non ai documenti, per questo varia di

qualche decina di millisecondi tra un'esecuzione e l'altra. Poiché Lucene è append-only (non esiste un update, solo delete e add), all'inizio dell'indicizzazione vengono cancellati i documenti già presenti (`deleteAll()`) per non duplicarli a ogni esecuzione.

4 Sintassi delle query

```
nome <termini>
  # es. nome roma
contenuto <termini>
  # es. contenuto "selva oscura"
exit
  # termina il programma
```

La prima parola indica il campo; il resto viene passato al `QueryParser` di Lucene con lo stesso analyzer dell'indicizzazione:

- termini senza modificatori → **opzionali** (OR)
- `+termine` → obbligatorio, `-termine` → escluso
- `"…"` → **phrase query**

Se la prima parola non è `nome` o `contenuto` viene mostrato un errore. Il ranking dei risultati è quello di default di Lucene (**BM25**).

5 Query di test

#	Query	Query Lucene	Risultati	Cosa verifica
1	`nome roma`	`nome:roma`	**storia_di_roma**	termine singolo, split su `_`
2	`nome carbonara`	`nome:carbonara`	**Ricetta Carbonara**	maiuscole e spazio nel nome
3	`nome "motori di ricerca"`	`nome:"motori di ricerca"`	**motori-di-ricerca**	phrase query sul nome
4	`nome intelligenza artificiale`	`nome:intelligenza nome:artificiale`	**intelligenza_artificiale**	più termini in OR
5	`contenuto pianeti`	`contenuto:pianet`	**sistema_solare**	stemming
6	`contenuto "selva oscura"`	`contenuto:"selva oscur"`	**divina_commedia**	phrase query
7	`contenuto +guanciale +pecorino`	`+contenuto:guancial +contenuto:pecorin`	**Ricetta Carbonara**	termini obbligatori
8	`contenuto impero romano`	`contenuto:imper contenuto:roman`	**impero-romano (1.89)** **storia_di_roma (1.21)** **Ricetta Carbonara (0.83)**	ranking BM25
9	`contenuto "indice invertito"`	`contenuto:"indic invertit"`	**motori-di-ricerca**	phrase query con stemming
10	`contenuto +lucene +java`	`+contenuto:lucen +contenuto:java`	**lucene_introduzione**	termini obbligatori
11	`contenuto il di la`	*(vuota, solo stopword)*	*nessun risultato*	rimozione stopword
12	`nome "ricerca motori"`	`nome:"ricerca motori"`	*nessun risultato*	phrase query: l'ordine conta

13	`titolo roma`	–	**errore: campo non valido**	gestione errori

Osservazioni.

- `nome roma` non restituisce `impero-romano` : sul nome non c'è stemming, quindi `roma` ≠ `romano` (scelta voluta; `nome romano` trova infatti solo `impero-romano`).
- Nella query 8 compare anche la carbonara per "pecorino romano": lo stemming porta "romano"/"romana" alla stessa radice `roman` , ma con score più basso perché il termine è meno frequente nel documento.
- Nella query 9 lo stemming viene applicato anche ai termini della frase, coerentemente con quanto fatto in indicizzazione.

Le query sono verificate automaticamente da `mvn test` : **11 test · 0 falliti**